

Dossiê de Funcionamento - MYINC Social Media AI

V2

Versão revisada: v2.0.0-local-engine | Arquitetura em 3 camadas | Data: 15/06/2026

Este dossiê descreve como a aplicação funciona, quais componentes foram revisados, quais garantias técnicas existem e quais validações ainda dependem do ambiente real de produção.

1. Resumo executivo

- A V2 foi organizada como aplicação nova e independente em três camadas: web na Vercel, Supabase como centro operacional e EXE local como motor pesado.
- O frontend não processa imagem, vídeo ou chamadas OpenAI. Ele apenas cria, aprova, agenda e monitora.
- O Supabase guarda dados, storage, fila, logs, prompts, regras da marca e executa apenas Edge Functions leves.
- O EXE local claima jobs, chama OpenAI usando chave local, processa um job por vez e devolve resultado por Edge Function segura.
- O publicador agendado não gera mídia. Ele apenas publica mídia pronta no Instagram/Meta e registra logs.

2. Arquitetura final

Camada	Responsabilidade	O que não faz
Frontend Web (Vercel)	Login, painel, planejamento, aprovação, memória, cérebro IA, biblioteca, estúdio, calendário, fila, logs e configurações.	Não usa OpenAI, não tem service role, não processa mídia pesada.
Supabase	Banco, Auth, Storage, fila, prompt payload, logs, RLS, RPCs e Edge Functions leves.	Não gera 30-40 imagens em request HTTP e não apaga secrets.
Motor Local EXE	Processa IA pesada, chama OpenAI, salva resultado e faz heartbeat.	Não decide estratégia e não usa service role.
Publicador leve	Publica posts agendados com mídia pronta via Meta Graph API.	Não gera IA e não simula sucesso.

3. Fluxo operacional completo

- Usuário entra no painel web com Supabase Auth.
- Usuário cria campanha de teste, aprova posts ou usa posts existentes.
- Painel chama enqueue-generation, uma Edge Function leve.
- Supabase monta o prompt com build_prompt_payload usando marca, perfil, regras, templates, biblioteca e contexto do post.
- São criados jobs em generation_jobs: content, image, carousel_page, video ou quality_check.
- EXE local registra worker, envia heartbeat, claima 1 job por vez com lock e processa.
- EXE envia resultado para engine-save-result; a função salva no Storage, cria media_assets, post_versions, atualiza posts e completa o job.
- Usuário revisa no Estúdio e agenda no Calendário.
- Publicador publish-scheduled-posts roda por cron, publica mídia pronta e grava publish_logs.

4. Supabase - estrutura criada

Scripts em supabase/v2 devem ser aplicados em ordem:

```
00_BACKUP_RUNTIME_SECRETS.sql
01_DROP_APP_TABLES_ONLY.sql
02_CREATE_V2_SCHEMA.sql
03_CREATE_RLS_POLICIES.sql
04_CREATE_STORAGE_BUCKETS.sql
05_SEED_MYINC_BRAIN.sql
06_CREATE_RPC_QUEUE_FUNCTIONS.sql
07_CREATE_PUBLISH_SCHEDULE.sql
```

- O backup preserva runtime_secrets quando a tabela existe.
- O reset remove somente tabelas antigas do app; não usa drop schema public cascade.
- O schema cria marcas, perfis, regras, templates, campanhas, posts, versões, assets, fila, workers, publicação, logs e settings.
- RLS é habilitado em todas as tabelas; worker grava por RPC/Edge segura.
- Buckets: creative-media, brand-assets e library.

5. Edge Functions leves

- enqueue-generation: valida usuário e enfileira jobs.
- build-prompt: monta payload de prompt sem gerar imagem.
- engine-register-worker: registra o EXE com chave de worker.
- engine-save-result: recebe resultado do EXE, salva mídia e completa job com service role no backend.
- publish-scheduled-posts: publica mídia pronta no Instagram/Meta; suporta imagem, carrossel e vídeo/Reels quando a Meta aceitar o formato.
- admin-status: mostra status seguro sem vaziar secrets.
- Correção aplicada: todas as Edge Functions respondem OPTIONS com cabeçalhos CORS corretos.

6. Motor Local EXE

- Roda em engine/runtime/main-loop.mjs.
- Lê .env.engine local no computador do operador.
- Registra worker via engine-register-worker.
- Usa claim_next_generation_job com lock e for update skip locked.
- Processa um job por vez.
- Chama OpenAI localmente para texto e imagem.
- Nunca usa SUPABASE_SERVICE_ROLE_KEY.
- Se vídeo não tiver provider real, falha com erro claro. Não existe sucesso simulado.

7. Aplicativo desktop/Electron

- Inclui main.cjs, tray.cjs, autostart.cjs e engine-process.cjs.
- Correção aplicada: inicialização do motor com ELECTRON_RUN_AS_NODE.
- Correção aplicada: ícone de tray e chamada real de tray no main.
- Ao fechar a janela, o app minimiza e mantém o motor rodando, salvo quando o usuário sair pelo menu.

8. Frontend Web

- App em apps/web com Vite + React.
- Telas: Dashboard, Planejamento, Aprovação, Memória, Cérebro, Biblioteca, Estúdio, Calendário, Fila, Motor, Logs e Configurações.
- Correção aplicada: nomes de secrets removidos do bundle frontend.
- Correção aplicada: edição básica da memória, edição/toggle de regras do cérebro IA e ações de retry/cancel na fila.
- O frontend cria jobs; não processa jobs.

9. Segurança

- Chave OpenAI fica somente no PC do motor local.
- Service role fica somente nas Edge Functions do Supabase.
- Frontend usa apenas URL pública e anon key.

- runtime_secrets é preservada e não tem select público.
- Worker precisa de chave local, enviada em header próprio e validada por hash.
- Publicador exige PUBLISH_CRON_SECRET; sem ele a função bloqueia execução.

10. Testes executados nesta revisão

```
npm run v2:verify
```

- engine:check passou: sintaxe do motor principal, cliente OpenAI, cliente Supabase e status-store validada.
- v2-architecture ok: camadas e workspace validados.
- v2-no-secret-leak ok: frontend sem nomes de secrets proibidas.
- v2-sql-contract ok: contratos críticos SQL validados.
- v2-engine-contract ok: motor claima/salva resultado e Electron inicia como Node.
- v2-edge-contract ok: Edge Functions com CORS, publicador com carrossel/vídeo e cron secret obrigatório.

11. Pontos que dependem do ambiente real

- Não é honesto prometer 100% produção sem aplicar SQLs no Supabase real e testar com suas chaves reais.
- É necessário criar usuário no Supabase Auth para login.
- É necessário configurar Edge Secrets do Supabase.
- É necessário configurar .env.engine no computador local.
- É necessário que a conta OpenAI tenha acesso aos modelos configurados.
- É necessário que o app Meta tenha permissões de publicação e Instagram Business conectado.
- Publicação de vídeo depende das regras e processamento assíncrono da própria Meta.

12. Checklist de validação em produção

- Rodar SQLs 00 a 07 em ordem.
- Criar usuário Supabase Auth e confirmar login no painel.
- Deploy das Edge Functions.
- Configurar Edge Secrets, Worker key, Publish cron secret, Meta tokens e IDs.
- Deploy web na Vercel com VITE_SUPABASE_URL e VITE_SUPABASE_ANON_KEY.
- Configurar .env.engine no PC local.
- Rodar npm run engine:dev e verificar worker online.
- Criar campanha teste com 5 posts.
- Enviar para produção e confirmar jobs em generation_jobs.
- Confirmar que o EXE claima 1 job, salva mídia no Storage e cria media_assets/post_versions.
- Aprovar post no Estúdio.
- Criar publish_queue e testar publicador com cron secret.
- Conferir publish_logs e system_logs.

Conclusão

A aplicação está estruturalmente correta para a arquitetura em 3 camadas. A revisão corrigiu falhas críticas antes de entrega: segredo no frontend, CORS, instalação por workspace, Electron/tray e publicador. O pacote é uma base sólida para aplicar no GitHub/Supabase e validar com ambiente real.